

NATIONAL TEACHERS COLLEGE
School of Arts, Sciences and Technology**STUDENT DOCUMENTATION WORKSHEET****NTC_CC105 – Data Structure w/Lab | Terminal Assessment, Weeks 1–5**

“Smart Campus Student Resource Management System”

This worksheet accompanies the NTC_CC105 Terminal Assessment (Weeks 1–5). Complete every section as a group, then submit this worksheet together with your source code, GitHub repository, and screenshots. Refer to the Terminal Assessment document for full requirement details.

Group No.: _____**Section:** BSIT 2.4**Date:** September 20, 2026**Instructor:** Prof. Darwin C. Vargas, MIT**Group Members and Assigned Positions**

No.	Full Name	Assigned Position
1	Mier, Ronamae	Project Lead / System Integrator
2	Quintela, Nicole M.	Data Structure Developer
3	Serdenia, Sheayla Rihanna A.	Memory & Pointer Developer
4	Cunanan, Jhoey	Matrix & Algorithm Developer
5	De Pedro, Precious Mae M.	Documentation & QA Lead

SECTION 6 – ALGORITHM DOCUMENTATION (Part VII)

Complete this table for each major operation in your system.

Operation	Input	Process	Output	Algorithm	Time Complexity
Student Insertion	Student record details (ID, Name, Section, Year Level)	Validate ID format, check for duplicate ID (scan all existing records), check the array isn't full, check no fields are empty, then write to studentArray[studentCount] and increment count	Updated student array with new record added (or an error message if any validation fails)	Duplicate-ID linear scan, then append at next free index	$O(n)$ dominated by the duplicate-ID scan (isDuplicateStudentID); the append itself is $O(1)$
Student Search	Target Student ID	Loop through the array from index 0, comparing each studentID to the target until a match or the end is reached	Matching record displayed, or "Record not found"	Linear search (findStudentIndex ByID)	$O(n)$
Student Update	Target Student ID, New info	Locate the record via linear search, then overwrite its section and yearLevel fields directly	Updated record in place (or "Student ID not found")	Search then in-place update	$O(n)$
Student Deletion	Target Student ID	Locate the record via linear search, then shift every following element one position left to close the gap, decrement count	Updated array, record removed, count reduced by 1	Search then shift-left deletion	$O(n)$
Array Traversal	Student array (or resource array)	Visit every active element once, from start to end, printing its fields	Displayed list of all active records.	Sequential traversal (for-loop)	$O(n)$
Matrix Processing	2D resource utilization matrix (4 departments $\times$ 4 resources)	Nested loop, for each row, iterate through every column, summing or displaying values	Row totals and column totals displayed	Nested loop traversal	$O(r \times c)$, which is $O(n^2)$ here since it's a 4×4 (square) matrix

SECTION 7 – BIG-O ANALYSIS (Part VIII)

Explain, in your own words, why each stated complexity applies to your implementation.

Operation	Time Complexity	Explain Why
Direct array access	$O(1)$	Arrays store elements in contiguous memory, allowing the program to access <code>studentArray[i]</code> or <code>studentMatrix[i][j]</code> directly using their indexes. Since the requested element can be accessed without checking the other elements, the operation takes constant time, $O(1)$, regardless of the array size.
Array traversal	$O(n)$	The program uses a loop to go through the student records one by one. If there are more students, the number of operations increases proportionally.
Linear search	$O(n)$	The <code>findStudentIndexById()</code> function uses a linear search to examine the records of the students in sequential order, beginning with the first record. Then it compares the stored student ID with the ID that is being searched for. If a match is found, the corresponding index is returned. However, if no match is found, the function continues until all n records have been checked. So, the worst-case time complexity is $O(n)$.
Matrix traversal ($n \times n$)	$O(n^2)$	The matrix functions use nested loops to process each of the rows and columns of the 2D array. Each element is examined during the operation, so an $n \times n$ matrix requires n^2 iterations. So, the matrix traversal has a time complexity of $O(n^2)$.